Image-to-Cash

Architecture and design rationale for automating Fakturama from an order image

Marwan Abdellah

November 2026

Abstract

Image-to-Cash turns a single order image (or PDF) into a saved, verified *Order* and a linked, paid *Invoice* in the Fakturama 2.2.0 desktop application. It drives the real user interface through Microsoft UI Automation (UIA) and OCR, without stored coordinates, and checks every step against Fakturama's own database. This note explains the architecture, the principles behind it, and how the design was hardened by the Elephant–Goldfish review method (EGM) and by live calibration. Source: `https://github.com/MarwanAbdellah/TJM-Fakturama-Automation`.

1 The task and its constraints

The brief asks for "one continuous Order-first flow": extract the order, open a *New Order*, use the Order's own Debtor and Product selectors as the existence checks, create missing master data only when an exact selection is unavailable, return to the same open Order, complete it, create the linked Invoice from the Order's follow-up area, apply the payment status, and verify the saved records—"without relying on hardcoded coordinates or a fixed UI layout", using UIA, OCR and/or LLMs. Four constraints shaped everything that follows:

1. **The UI is the only write path.** Fakturama is an Eclipse RCP/SWT application; its item table is a NatTable that UIA cannot read cell by cell.
2. **Every step must be verified** before moving on (brief §1–§5), including the persisted payment state.
3. **Wrong data is worse than no data.** A duplicate Debtor or a mispriced line costs more than a stopped run.
4. **The flow is fixed by the brief.** A faster "masters-first" redesign (pre-create all products, then fill the Order) was drafted and reviewed, then deliberately dropped to keep the brief's Order-first semantics.

2 Architecture

The system is a pipeline with one data contract, `OrderDraft` (a Pydantic model), between reading and acting (Figure 1). Reading never touches Fakturama; acting never re-reads the image.

Extraction.
A local OCR engine (Indra) feeds a parser that anchors on printed labels rather than positions, so the template survives shifts and rescans. For layouts it cannot read, an *opt-in* and still *experimental* vision model (any OpenAI-compatible endpoint; OpenRouter by default) fills the same schema. It is off by default because the image leaves the machine; its output gets no special trust: the same checks, a second reading told exactly which checks failed, and human review.

Control grounding.
Controls are found by a ladder, cheapest and most stable first: (1) the UIA name (toolbar tooltips, navigation links, totals); (2) a *label anchor*, the nearest field right of or below a static label; (3) a

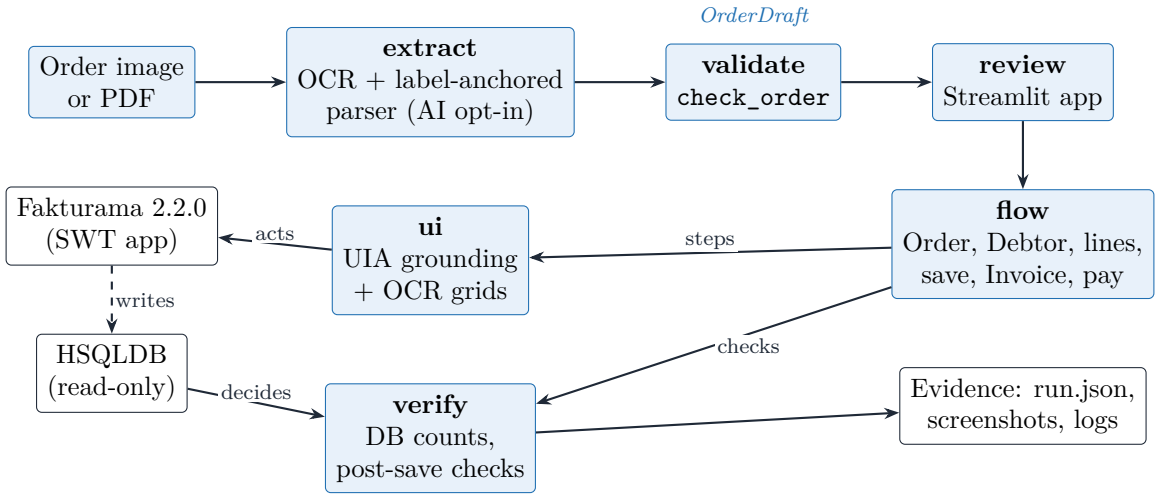


Figure 1: Pipeline. Reading (top) never touches Fakturama; the UI layer acts; the read-only database decides and verifies; every run leaves an evidence trail. `OrderDraft` is the one contract between reading and acting.

combo identified by the options it offers; (4) OCR of the NatTable grid, with columns taken from the *drawn* grid lines and named by a majority vote of the readable headers.

Decision and verification.
Fakturama's HSQLDB is opened strictly read-only over JDBC. It answers the questions OCR must not: "does this SKU exist exactly once?", "was the Debtor saved with both address roles?", "is the Invoice linked to this Order, paid, with this date and value?"

Operation.
A Streamlit app reviews drafts, previews what will be selected or created, books single orders or a batch, and pages through each run's steps and screenshots. A CLI does the same headless.

3 The mind behind it: design principles

Principle	What it means in the code
OCR locates, the DB decides	An "exact match" is a database count: 0 ⇒ create, 1 ⇒ select, ≥2 ⇒ stop. A grid row that looks right but is not confirmed by the DB stops the run.
Stop, don't guess	Two failure types: `ManualReview` (the data is ambiguous; a human decides) and `AutomationError` (the UI did not reach the expected state). Neither is retried silently.
Read back every write	Every typed value is read back; every line is confirmed by the running *Total Net*; every save is confirmed by a new database row.
Prices come from the catalogue	Product prices are never typed into the Order; a catalogue price that differs from the order stops the run instead of being overwritten.
Idempotent and resumable	A reference that already has documents is never booked twice; a saved Order without an Invoice resumes at the Invoice step after being re-verified against the image.
Safe by construction	The test Fakturama is driven if open, otherwise the single open one; either way it must own the database port, so checks never read another instance's data. One run at a time (file lock), never close an unsaved tab, no machine-specific paths in code.

4 How the design was hardened

Elephant–Goldfish Method (EGM). Each design was reviewed with EGM: the *elephant* (the drafting agent, with full context: brief, prototype, live UIA probes) writes the design; *goldfish* reviewers with zero context (design critic, comprehension, implementation readiness, code details, zoom-out/simplicity) read only the artefacts, and a synthesizer merges, de-duplicates and re-checks their findings. Every finding is explicitly accepted, adapted or rejected, and every UI mechanic a reviewer called unproven is spiked against the live application before it is built.

- **Design round:** 14 findings (7 high, 5 medium, 2 low). Examples: "exact match decided by OCR on truncated cells" ⇒ the DB-count rule above; "SWT date fields ignore plain text" ⇒ a segmented date-field driver; "payment branch under-specified" ⇒ the method is ensured *before* New Debtor, because the Debtor editor only loads its payment list when it opens.
- **Whole-repository round (v4):** 5 high, 23 medium, 31 low. The most important high finding (B-H1): the saved documents were never checked against the debtor actually chosen. It was fixed by recording the chosen contact and asserting it on every receiver row of the saved Order and Invoice; all five high findings were closed.

What only the live application revealed. No reviewer predicted these; calibration runs did. Saving while the dynamic *Delivery address* tab is active throws a NullPointerException (also visible in the brief's own figures), so the Invoice-address tab is selected before every save. Selectors auto-accept as soon as one row remains, so search terms are pasted whole. The VAT field parses "." while price fields follow the locale ("6,25" became 625 %). Fakturama raises its own *Duplicate Contact* dialog on same name and street, so that rule is now checked in the database first. OCR can join a right-aligned quantity to the adjacent SKU, which is repaired before lines are compared.

Evidence of quality. 144 unit tests cover the business rules and the decision points (select-or-create, post-save checks, batch policy, AI-fallback retries) with the UI and database stubbed; the pre-push hook runs them and `ruff`. Live runs on the test workspace booked the template orders end to end and produced a step log and annotated screenshots for each run. Measurement drove optimisation: an 8-line order spent about 120 s of 264 s on repeated grid OCR, which led to one grid read per line and column-wise filling.

5 Limits and next steps

- Batch mode retries a stopped order once, skips orders already booked, and pauses on Esc; a stopped run leaves its unsaved Order tab for a human.
- Only page 1 of a PDF is read; order-level discount and shipping are not mapped; foreign-currency orders stop.
- OCR of very long, scrolling item tables is the slowest and least certain step.

With three more hours. The first priority would be to make the AI service more robust. Today it is an opt-in fallback for layouts the local parser cannot read: one structured-output call to an OpenAI-compatible model (OpenRouter by default), retried on busy replies and asked once more with the problems the checks found. To trust it more: run it on every order and compare it field by field with the local OCR, flagging disagreements for review; measure it on a labelled set of unseen layouts before relying on it; let it fall back to a local vision model when the service is unavailable, so images can stay on the machine; and keep its answers in the run evidence.

The second would be a reproducible environment, so anyone gets the same setup with one command. Docker fits the parts that need no screen: OCR, parsing, the checks, the AI reader and the unit tests, pinned in an image with the OCR models baked in. Docker cannot drive Fakturama itself, because UI Automation needs a real, unlocked Windows desktop session, which a container does not have. For the

end-to-end runs that means a scripted Windows virtual machine (or a Windows Sandbox configuration) with Fakturama, a test workspace already set to EUR and this repository installed, restored to the same clean state before every run. After that: a generic watcher for unexpected dialogs, cleanup of a failed run's records, and running a whole queue in one process.